Program Execution Flow Reference

STARTUP — Part 1 of 2 (runs once at power-on)

Entry / JVM | Every 20ms loop | Once per mode change | CommandScheduler | AdvantageKit | Hardware / IO | Config / constants | Framework-managed

Main.java
`main()`

↓

```
RobotBase.startRobot(Robot::new)
// framework creates Robot instance
// enters TimedRobot 20ms Notifier loop
```

↓

Robot() constructor
// runs once at power-on

calls →

```
Logger.recordMetadata("ProjectName","BearBotsXRP")
Logger.addDataReceiver(new WPILOGWriter())
// writes .wpilog file to USB drive
Logger.addDataReceiver(new NT4Publisher())
// streams live to AdvantageScope via NT4
Logger.start()
```

↩ returns

AKit begins recording all inputs from this point forward.

↓

new RobotContainer()
// instantiates all subsystems

calls →

```
switch (Constants.currentMode) {
  case REAL:
    drive = new Drive(new DriveIOXRP())
    arm   = new Arm(new ArmIOXRP())
  case SIM:
    drive = new Drive(new DriveIOSim())
    arm   = new Arm(new ArmIOSim())
  case REPLAY:  // no hardware
    drive = new Drive(new DriveIO() {})
    arm   = new Arm(new ArmIO() {})
}
```

↩ returns

IO impl chosen once here. Subsystems never know which hardware they get.

↓

configureButtonBindings()
// registers triggers — one time only

calls →

```
// Registers triggers with CommandScheduler
// Does NOT call commands yet
// Scheduler polls these every loop at step 2
controller.a().onTrue(
  arm.setAngleCommand(kHighAngleDeg))
controller.b().onTrue(
  arm.setAngleCommand(kLowAngleDeg))
controller.x().onTrue(arm.runOnce(arm::stow))
drive.setDefaultCommand(
  drive.defaultDriveCommand(controller))
```

↩ returns

Registration only. Scheduler checks button state every loop at step 2.

↓

configureAutonomous() continues on next page →

Program Execution Flow Reference

STARTUP — Part 2 of 2, then 20ms LOOP — Part 1 of 2

Entry / JVM | Every 20ms loop | Once per mode change | CommandScheduler | AdvantageKit | Hardware / IO | Config / constants | Framework-managed

← continuing from previous page: configureButtonBindings()

```
configureAutonomous()
// registers auto modes — one time only
```

calls → / ← returns

```
autoChooser.addOption("Drive Forward", driveForwardCmd)
autoChooser.addOption("Spin in Place", spinCmd)
SmartDashboard.putData("Auto Chooser", autoChooser)
// Driver selects before match via dashboard
// Retrieved in Robot.autonomousInit()
```

```
DS mode state machine
// WPILib checks DS state each loop
// init() fires only when mode changes
```

calls → / ← returns

```
disabledInit()  // on → disabled
  // runs only the code you write here (e.g. stop motors)
autonomousInit()  // on → autonomous
  autonomousCommand =
    robotContainer.getAutonomousCommand()
  autonomousCommand.schedule()
teleopInit()  // on → teleop
  if (autonomousCommand != null)
    autonomousCommand.cancel()
```

Each init() fires ONCE per transition. Most loops nothing fires here.

STARTUP COMPLETE

20ms LOOP — Notifier fires 50× per second

```
Notifier fires  →  loopFunc()
// every 20ms — 50× per second
// START of one loop cycle
```

```
loopFunc(): DS mode changed?
// most loops: no change → skip
```

calls → / ← returns

```
// IF mode changed — fires ONCE per transition:
disabledInit() // runs only the code you write here (e.g. stop
motors)
autonomousInit() // schedule auto command
teleopInit()     // cancel auto command
// Button triggers from configureButtonBindings()
// are polled by CommandScheduler at step 2 below
```

Most loops nothing fires here. Init methods are one-shot.

```
modePeriodic()  ← current mode only
// exactly one fires per loop
```

calls → / ← returns

```
disabledPeriodic()
  // update auto chooser from dashboard
autonomousPeriodic()
  // usually empty in command-based code
  // framework still calls this every teleop loop
teleopPeriodic()
  // usually empty in command-based code
  // framework still calls this every auto loop
```

CommandScheduler runs because Robot.robotPeriodic() calls CommandScheduler.getInstance().run().

Robot.robotPeriodic() continues on next page →

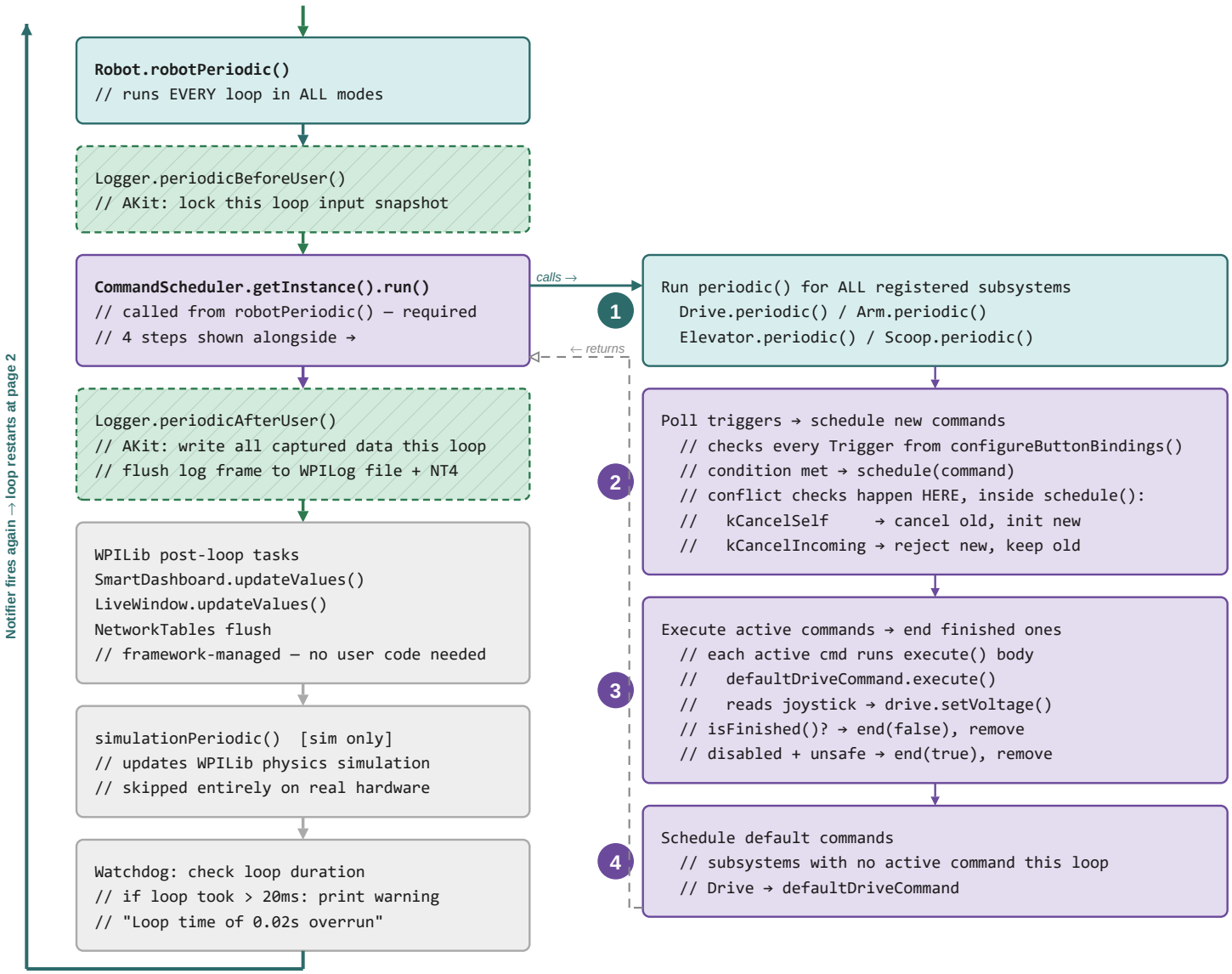
loop cycles back here — Notifier fires every 20ms

Program Execution Flow Reference

20ms LOOP — Part 2 of 2 (robotPeriodic through watchdog)

Entry / JVM | Every 20ms loop | Once per mode change | CommandScheduler | AdvantageKit | Hardware / IO | Config / constants | Framework-managed

← continuing from previous page: modePeriodic()



Program Execution Flow Reference

CALL DETAIL — inside CommandScheduler steps 1 (periodic) and 3 (execute)

Entry / JVM | Every 20ms loop | Once per mode change | CommandScheduler | AdvantageKit logging | Hardware / IO layer | Config / constants | Framework-managed

1 Arm.periodic() ← called by scheduler
// step 1 for every registered subsystem

calls → / ← returns

```
io.updateInputs(inputs)
// IO impl reads hardware → fills struct
```

calls → / ← returns

```
ArmIOXRP.updateInputs():
  inputs.commandedAngleDeg = servo.getAngle()
DriveIOXRP.updateInputs():
inputs.leftPositionM = leftEncoder.getDistance()
inputs.rightPositionM =
rightEncoder.getDistance()
  inputs.gyroAngleDeg = gyro.getAngle()
```

calls → / ← returns

```
XRP Hardware reads:
XRPServo(5).getAngle()  // Servo 2, port 5
Encoder(4,5).getDistance()  // left DIO 4/5
XRPGyro.getAngle()
```

inputs should now be treated as this cycle"s snapshot.□Do not modify inputs directly after processInputs().

```
Logger.processInputs("Arm", inputs)
// REAL: writes to log REPLAY: reads from
log
```

calls → / ← returns

```
// REAL mode:
//   inputs written to .wpilog file
// REPLAY mode:
//   inputs overwritten FROM .wpilog file
//   subsystem sees identical values as real match
// This is the key AKit replay guarantee
```

processInputs() = replay-critical hardware data. recordOutput() = calculated values and visualization.

```
Arm.periodic() continues
armLigament.setAngle(180 -
inputs.commandedAngleDeg)
Logger.recordOutput("Arm/Mechanism2d",
mechanism)
// pose collected later by
Robot.robotPeriodic() →
```

calls → / ← returns

```
// 180- flips coordinate convention
// 2D stick-figure in AdvantageScope
getArmPose3d() returns:
  new Pose3d(-0.052, 0.007, 0.0645,
    new Rotation3d(0,
      toRadians(inputs.commandedAngleDeg), 0))
// Robot.robotPeriodic() combines this with
// drive.getDrivePose3d() into ONE
// "FinalComponentPoses" array — avoids two
// subsystems overwriting the same AKit key.
```

3 defaultDriveCommand.execute()
// called by scheduler for active commands

calls → / ← returns

```
double speed = -controller.getLeftY()
double turn  =  controller.getRightX()
drive.setVoltage(
  (speed+turn) * kMaxBatteryVoltage,
  (speed-turn) * kMaxBatteryVoltage)
```

calls → / ← returns

```
DriveIOXRP.setVoltage(leftV, rightV)
leftMotor.set(clamp(leftV / kMaxBatteryVoltage,
-1, 1))
rightMotor.set(clamp(rightV / kMaxBatteryVoltage,
-1, 1))
// kMaxBatteryVoltage = 4.5 V
// rightMotor.setInverted(true) was set ONCE in
the
// constructor — do NOT also negate pct here, or
the
// right side double-inverts and drives backward.
```

calls → / ← returns

```
XRP Hardware writes:
XRPMotor(0).set(pct)  // left motor
XRPMotor(1).set(pct) // right — already
inverted
// via setInverted(true)
```

```
arm command execute()  (if button held)
```

calls → / ← returns

```
arm.setAngle(angleDeg)
io.setAngle(angleDeg)
Logger.recordOutput(
  "Arm/Commanded/AngleDeg", angleDeg)
```

calls → / ← returns

```
ArmIOXRP.setAngle(angleDeg)
servo.setAngle(angleDeg)
// XRPServo port 5
```

calls → / ← returns

```
XRP Hardware write:
XRPServo(5).setAngle(deg)
// Servo 2 on XRP board
// physical arm moves
```

Program Execution Flow Reference

FILE CARDS — 1 of 10

Main.java

JVM entry

frc/robot/

main()

JVM entry

```
RobotBase.startRobot(Robot::new)
// JVM entry point — OS calls this once at launch
// framework instantiates Robot, then enters
// the 20ms TimedRobot Notifier loop
```

Program Execution Flow Reference

FILE CARDS — 2 of 10

Robot.java

50 Hz

frc/robot/

Robot() // constructor — once

```
Logger.recordMetadata("ProjectName","BearBotsXRP")
Logger.addDataReceiver(new WPILOGWriter())
// → writes .wpilog file to USB drive
Logger.addDataReceiver(new NT4Publisher())
// → streams live data to NetworkTables
Logger.start()
// AKit starts capturing — must be called before
// RobotContainer so all subsystems are logged
robotContainer = new RobotContainer()
```

robotPeriodic() // ALL modes, every loop — 50 Hz

```
// LoggedRobot wraps this call automatically — not called by us:
Logger.periodicBeforeUser()  //   framework-managed
// AKit: locks this loop's input snapshot
CommandScheduler.getInstance().run()
// runs all scheduler steps — this line IS ours
// LoggedRobot wraps this call automatically — not called by us:
Logger.periodicAfterUser()  //   framework-managed
// AKit: flush log frame to file + NT4
// Combine poses ONCE here (not per-subsystem) — avoids
// two subsystems overwriting the same AKit key:
Logger.recordOutput("FinalComponentPoses",
    new Pose3d[]{ drive.getDrivePose3d(), arm.getArmPose3d() })
```

autonomousInit() — once

```
autonomousCommand = robotContainer.getAutonomousCommand()
if (autonomousCommand != null)
    autonomousCommand.schedule()
```

teleopInit() — once

```
if (autonomousCommand != null)
    autonomousCommand.cancel()
// stops auto command if still running
```

disabledInit() — once

```
// Fires ONCE on transition. Runs ONLY code you write here —
// nothing automatic (no auto motor stop). e.g. call
// subsystem stop()/reset() or cancelAll() if desired.
// Separately, EVERY LOOP while disabled, the scheduler ends
// any active command not marked runsWhenDisabled() — that
// ongoing check is NOT part of disabledInit().
```

disabledPeriodic() — 50 Hz

```
// update auto chooser selection from SmartDashboard
```

Program Execution Flow Reference

FILE CARDS — 3 of 10

RobotContainer.java

once

frc/robot/

RobotContainer() // constructor — once

```
switch (Constants.currentMode) {
  case REAL:
    drive = new Drive(new DriveIOXRP())
    arm   = new Arm(new ArmIOXRP())
  case SIM:
    drive = new Drive(new DriveIOSim())
    arm   = new Arm(new ArmIOSim())
  case REPLAY:  // no hardware
    drive = new Drive(new DriveIO() {})
    arm   = new Arm(new ArmIO() {})
}
configureButtonBindings()
configureAutonomous()
```

configureButtonBindings() // registers once — once

```
// Registers triggers — does NOT call commands yet
// Scheduler polls these every loop at step 2
controller.a().onTrue(
    arm.setAngleCommand(ArmConstants.kHighAngleDeg))
controller.b().onTrue(
    arm.setAngleCommand(ArmConstants.kLowAngleDeg))
controller.x().onTrue(arm.runOnce(arm::stow))
drive.setDefaultCommand(
    drive.defaultDriveCommand(controller))
```

configureAutonomous() // registers once — once

```
autoChooser.addOption("Drive Forward", driveForwardCmd)
autoChooser.addOption("Spin in Place", spinCmd)
SmartDashboard.putData("Auto Chooser", autoChooser)
```

getAutonomousCommand() — getter

```
return autoChooser.getSelected()
// called by Robot.autonomousInit()
```

Program Execution Flow Reference

FILE CARDS — 4 of 10

Drive.java

50 Hz

frc/robot/subsystems/drive/

Drive(DriveIO io) // constructor — once

```
this.io = io
// SubsystemBase registers with CommandScheduler
// periodic() will be called every loop at step 1
```

periodic() // scheduler step 1 — 50 Hz

```
io.updateInputs(inputs)
// IO impl reads motors, encoders, gyro → fills inputs
Logger.processInputs("Drive", inputs)
// AKit logs inputs snapshot
Logger.recordOutput("Drive/LeftPositionM",  inputs.leftPositionM)
Logger.recordOutput("Drive/RightPositionM", inputs.rightPositionM)
Logger.recordOutput("Drive/GyroAngleDeg",   inputs.gyroAngleDeg)
// NOTE: does NOT write "FinalComponentPoses" directly —
// that key is aggregated once in Robot.robotPeriodic()
// (see Robot.java card) to avoid multiple subsystems
// overwriting the same AdvantageScope key.
```

getDrivePose3d() — getter

```
return new Pose3d(0, 0, 0,
    new Rotation3d(0, 0, toRadians(inputs.gyroAngleDeg)))
// Chassis pose for THIS subsystem only.
// Robot.robotPeriodic() collects all subsystem poses
// into ONE "FinalComponentPoses" array — see Robot.java card.
// Teaching simplification: this visualizes heading only.
// A full drivetrain would compute x/y pose from odometry.
```

setVoltage(double leftV, double rightV) — output

```
io.setVoltage(leftV, rightV)
// delegates to DriveIOXRP or DriveIOSim
Logger.recordOutput("Drive/Commanded/LeftV",  leftV)
Logger.recordOutput("Drive/Commanded/RightV", rightV)
```

stop() — output

```
setVoltage(0.0, 0.0)
```

defaultDriveCommand(XboxController ctrl) — getter

```
// Returns Command used as subsystem default
// Scheduler runs this when no other Drive cmd is active
return run(() -> {
    double speed = -ctrl.getLeftY()
    double turn  =  ctrl.getRightX()
    setVoltage((speed + turn) * kMaxBatteryVoltage,
               (speed - turn) * kMaxBatteryVoltage)
})
```

Program Execution Flow Reference

FILE CARDS — 5 of 10

Arm.java

50 Hz

frc/robot/subsystems/arm/

Arm(ArmIO io) // constructor — once

```
this.io = io
mechanism   = new LoggedMechanism2d(3, 3)
root        = mechanism.getRoot("ArmPivot", 1.2, 0.18)
armLigament = root.append(
    new LoggedMechanismLigament2d("Arm", feetToMeters(3), 0))
```

periodic() // scheduler step 1 — 50 Hz

```
io.updateInputs(inputs)
// reads servo state → inputs.commandedAngleDeg
Logger.processInputs("Arm", inputs)
// AKit logs inputs snapshot
armLigament.setAngle(180 - inputs.commandedAngleDeg)
// 180- flips coordinate convention for AdvantageScope
Logger.recordOutput("Arm/Mechanism2d", mechanism)
// 2D stick-figure diagram in AdvantageScope
// NOTE: does NOT write "FinalComponentPoses" directly —
// see getArmPose3d() below + Robot.robotPeriodic()
```

getArmPose3d() — getter

```
return new Pose3d(-0.052, 0.007, 0.0645,
    new Rotation3d(0, toRadians(inputs.commandedAngleDeg), 0))
// 3D joint pose for THIS subsystem only.
// Robot.robotPeriodic() collects all subsystem poses
// into ONE "FinalComponentPoses" array — see Robot.java card.
```

setAngle(double angleDeg) — output

```
io.setAngle(angleDeg)
// delegates to ArmIOXRP or ArmIOSim
Logger.recordOutput("Arm/Commanded/AngleDeg", angleDeg)
```

stow() — output

```
setAngle(ArmConstants.kStowedAngleDeg)
// safe stow = 180°
// servo holds position — not cutting power
```

getCommandedAngleDeg() — getter

```
return inputs.commandedAngleDeg
```

ArmIO.java

interface

frc/robot/subsystems/arm/

@AutoLog class ArmIOInputs — AKit

```
// @AutoLog generates ArmIOInputsAutoLogged at compile time
// Every field is auto-logged — no manual Logger calls needed
public double commandedAngleDeg = ArmConstants.kStowedAngleDeg
```

updateInputs(ArmIOInputs inputs) — contract

```
default void updateInputs(ArmIOInputs inputs) {}
// default = no-op (log replay stub)
// ArmIOXRP overrides:
//   inputs.commandedAngleDeg = servo.getAngle()
```

setAngle(double angleDeg) — contract

```
default void setAngle(double angleDeg) {}
// default = no-op (log replay stub)
// ArmIOXRP overrides:
//   servo.setAngle(angleDeg)
```

Program Execution Flow Reference

FILE CARDS — 7 of 10

ArmIOXRP.java

reads HW

frc/robot/subsystems/arm/

ArmIOXRP() // constructor — once

```
servo = new XRPServo(5)
// port 5 = Servo 2 on XRP board
```

updateInputs(ArmIOInputs inputs) — reads HW

```
inputs.commandedAngleDeg = servo.getAngle()
// reads current servo position from hardware
// called by Arm.periodic() at scheduler step 1
```

setAngle(double angleDeg) — writes HW

```
servo.setAngle(angleDeg)
// XRPServo.setAngle() sends PWM signal
// physical servo moves to requested angle
```

Program Execution Flow Reference

FILE CARDS — 8 of 10

DriveIOXRP.java

reads HW

frc/robot/subsystems/drive/

DriveIOXRP() // constructor — once

```
leftMotor    = new XRPMotor(0)   // port 0 = left motor
rightMotor   = new XRPMotor(1)   // port 1 = right motor
rightMotor.setInverted(true)     // right must be inverted
leftEncoder  = new Encoder(4, 5) // DIO pins 4/5
rightEncoder = new Encoder(6, 7) // DIO pins 6/7
gyro         = new XRPGyro()
```

updateInputs(DriveIOInputs inputs) — reads HW

```
inputs.leftPositionM     = leftEncoder.getDistance()
inputs.rightPositionM    = rightEncoder.getDistance()
inputs.leftVelocityMPS   = leftEncoder.getRate()
inputs.rightVelocityMPS  = rightEncoder.getRate()
inputs.gyroAngleDeg      = gyro.getAngle()
inputs.gyroRateDegPerSec = gyro.getRate()
```

setVoltage(double leftV, double rightV) — writes HW

```
leftMotor.set(MathUtil.clamp(
    leftV / Constants.kMaxBatteryVoltage, -1.0, 1.0))
rightMotor.set(MathUtil.clamp(
    rightV / Constants.kMaxBatteryVoltage, -1.0, 1.0))
// kMaxBatteryVoltage = 4.5 V  (4×AA XRP nominal)
// XRPMotor.set() takes percent output: -1.0 to 1.0
// (NOT 0.0–1.0 — motor needs negative values to reverse)
```

Constants.java

static

frc/robot/

RobotMode // enum — static

```
REAL,      // running on physical XRP hardware
SIM,       // WPILib desktop simulation
REPLAY     // log replay (no hardware, reads from log)
```

currentMode — static

```
public static final RobotMode currentMode = RobotMode.REAL
// Change to SIM for simulation, REPLAY for log replay
```

kMaxBatteryVoltage — static

```
public static final double kMaxBatteryVoltage = 4.5
// 4×AA alkaline cells — XRP nominal supply voltage
// DriveIOXRP divides commanded volts by this value
// to convert volts → -1.0 to 1.0 motor percent
```

Program Execution Flow Reference

FILE CARDS — 10 of 10

ArmConstants.java

static

frc/robot/subsystems/arm/

Servo angle limits static

```
kMinAngleDeg   =   0.0  // minimum allowed servo angle (degrees)
kMaxAngleDeg   = 180.0  // maximum allowed servo angle (degrees)
```

Named positions static

```
kStowedAngleDeg = 180.0  // safe / stowed arm position
kLowAngleDeg    = 135.0  // low position
kHighAngleDeg   =  90.0  // high position
```